



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

BACHELOR OF COMPUTER SCIENCE (DATA ENGINEERING) WITH HONS.

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3106 (S01) - APPLICATION DEVELOPMENT

FINAL TEST

TITLE : RESTAURANT WAITER ORDER SYSTEM

PREPARED BY :

NAME	MATRIC NO
NURUL ADRIANA BINTI KAMAL JEFRI	A23CS0258

PREPARED FOR:

DR. AHMAD NAJMI BIN AMERHAIDER NUAR

TABLE OF CONTENT

1.0 INTRODUCTION	3
2.0 SOFTWARE, LANGUAGE, TOOLS, API, TEMPLATES	3
3.0 DEVELOPMENT STEPS	4
3.1 Project Setup	4
3.2 Backend Project Creation	4
3.3 SDK Integration	4
3.4 Schema Creation	4
3.5 Menu Tab	5
3.6 Orders Tab	5
3.7 Order Detail Screen	5
3.8 Testing on the Emulator/Device	5
3.9 Screenshots and Report	5
4.0 SYSTEM INTERFACE	5
4.1 Screen 1 (Menu List)	5
4.2 Screen 2 (Add / Edit Menu Item)	6
4.3 Screen 3 (Orders List)	8
4.4 Screen 4 (New Order)	8
4.5 Screen 5 (Order Detail)	9
5.0 CONCLUSION	10

1.0 INTRODUCTION

The design and development of the Restaurant Waiter Order System is documented in this report, containing the thoroughness of this Flutter mobile application built for the Individual Test. The application enables a single user, the waiter, to manage the restaurant's menu and take orders from customers using a mobile device without any login or registration stage, as has been specified in the rubric.

The ultimate goal of the application is to display finalized CRUD (Create, Read, Update, Delete) functionality linked to a real cloud database, instead of local or in-memory storage. The waiter is able to add and edit menu items, mark items as available or not, create new orders for a specific table with multiple line items, and move each order through its full lifecycle from Pending to Paid.

The system is deliberately designed to have five screens and two top-level tabs, which are Menu and Orders, so that it remains simple to build, test, and illustrate within the test window, while also ensuring every CRUD operation needed by the requirement.

2.0 SOFTWARE, LANGUAGE, TOOLS, API, TEMPLATES

The following technologies and tools were used to build this project;

- Language: Dart
- Framework: Flutter
- IDE: Visual Studio Code with the Flutter and Dart extensions
- Emulator/Device: Android device connected via USB debugging (Android Studio's SDK platform-tools used for driver setup only)
- Backend: Supabase (PostgreSQL database with Row Level Security)
- Backend SDK: supabase_flutter (v2), using the Publishable API key for client-side access
- State management: setState (built-in Flutter state management, sufficient for this app's scope)
- UI components: Material 3 widgets, which are Scaffold, NavigationBar, ListView, Card, TextFormField, AlertDialog, ChoiceChip, SwitchListTile.
- Version control: Git and GitHub repository

Supabase was selected because it provides a real SQL (Postgres) database compared to Firebase, which naturally navigates onto the three-table relational data model needed by the brief (menu_items, orders, order_items connected by foreign keys).

3.0 DEVELOPMENT STEPS

This is the followed step-by-step method described in the test brief to build the application.

3.1 Project Setup

The Flutter project structure is created with the required folders (lib/models, lib/services, lib/screens) and runs the default app on the emulator/device to ensure the toolchain works before proceeding with any feature code.

3.2 Backend Project Creation

A new Supabase project is created in the Northeast East region, as it is close to Malaysia, for lower latency, and logged project URL and Publishable API key for use in the application.

3.3 SDK Integration

The supabase_flutter package is added via pubspec.yaml and Supabase.initialize() is called inside main() before runApp(), therefore the client is ready before any screen tries to query the database.

3.4 Schema Creation

A SQL script (schema_sql) in the Supabase SQL Editor is written and ran to generate the three required tables, which are menu_items, orders, and order_items with a foreign key from order_items to orders (ON DELETE CASCADE) and a foreign key to menu_items (ON DELETE SET NULL) so the past order data is saved even when a menu item is later unavailable or removed. On all three tables, Row Level Security was allowed with open policies, suitable since the instructions implied no authentication.

3.5 Menu Tab

The menu list screen is built (retrieve and show all menu items with name, price, category, and a switch of food availability), the add/edit form screen with validation on name, price, and category fields), and delete option with a confirmation dialog.

3.6 Orders Tab

The orders list screen with status filter chips is built (All, Pending, Preparing, Served, Paid), and the New Order screen where the waiter adds a table number and accommodates quantities for each available menu item using steppers, with a running total shown at the bottom.

3.7 Order Detail Screen

The order detail screen is built and shows each line item using the name and price snapshot taken at order time, the order total, and a button to update the order to its next status. Quantities can still be changed and the order can be cancelled (deleted) while the order status is still Pending.

3.8 Testing on the Emulator/Device

The full flow is tested; added a new menu item, created an order with 2 - 3 line items for a table, updated the order through Pending -> Preparing -> Served -> Paid, and deleted a menu item later to ensure existing orders still shown the correct past name and price through the snapshot fields.

3.9 Screenshots and Report

Each screen is screenshots (shown in Section 4), and compiled in this report.

4.0 SYSTEM INTERFACE

The following screens were captured from the live application, with one screenshot per required screen with a short description describing its functionality.

4.1 Screen 1 (Menu List)

The menu tab shows all menu items with their category, price, an availability switch, and edit/delete icons.

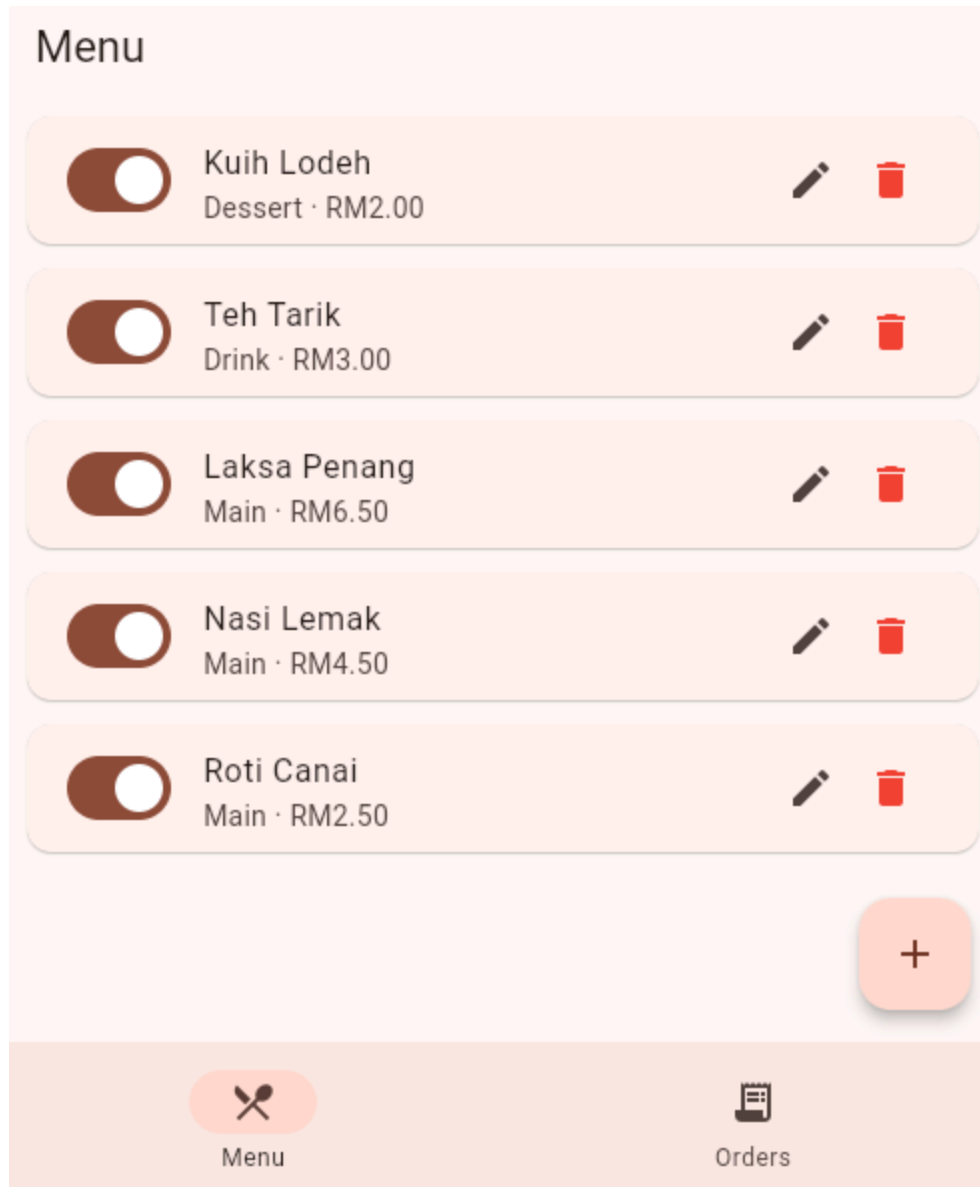


Figure 1 Menu List screen

4.2 Screen 2 (Add / Edit Menu Item)

The Add/Edit Menu Item form, with verified fields for name, price, and category, and an availability toggle.

 Add Menu Item

Name

Price (RM)

Category (e.g. Drink, Main, Dessert)

Available

☒

Save

***Figure 2** Add Menu Item screen*

 Edit Menu Item

Name

Laksa Penang

Price (RM)

6.5

Category (e.g. Drink, Main, Dessert)

Main

Available

☒

Save

***Figure 3** Edit Menu Item screen*

4.3 Screen 3 (Orders List)

The Orders tab with status filter chips, showing each order's table number, total, and current status.

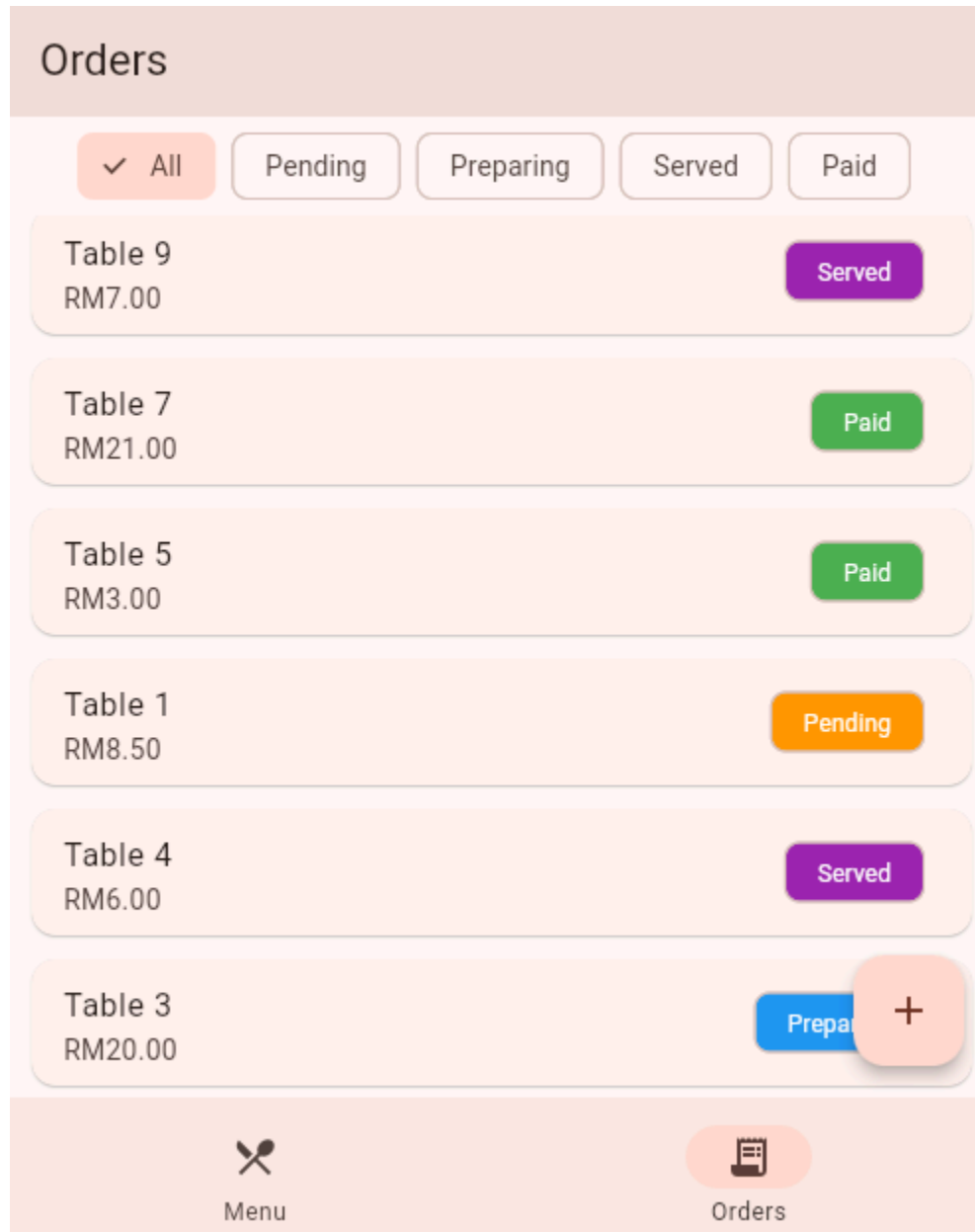


Figure 4 Order List screen

4.4 Screen 4 (New Order)

The New Order screen, displaying the table number field, quantity steppers for each available menu item, and the running total.

←

New Order

Table number

19

Menu items

Kuih Lodeh

RM2.00

⊖

0

⊕

Teh Tarik

RM3.00

⊖

1

⊕

Laksa Penang

RM6.50

⊖

0

⊕

Nasi Lemak

RM4.50

⊖

1

⊕

Roti Canai

RM2.50

⊖

0

⊕

Total: RM7.50

Place Order

Figure 5 OrderDetail screen

4.5 Screen 5 (Order Detail)

The Order Detail screen, displaying line items, the order total, and the button used to advance the order to its next status.

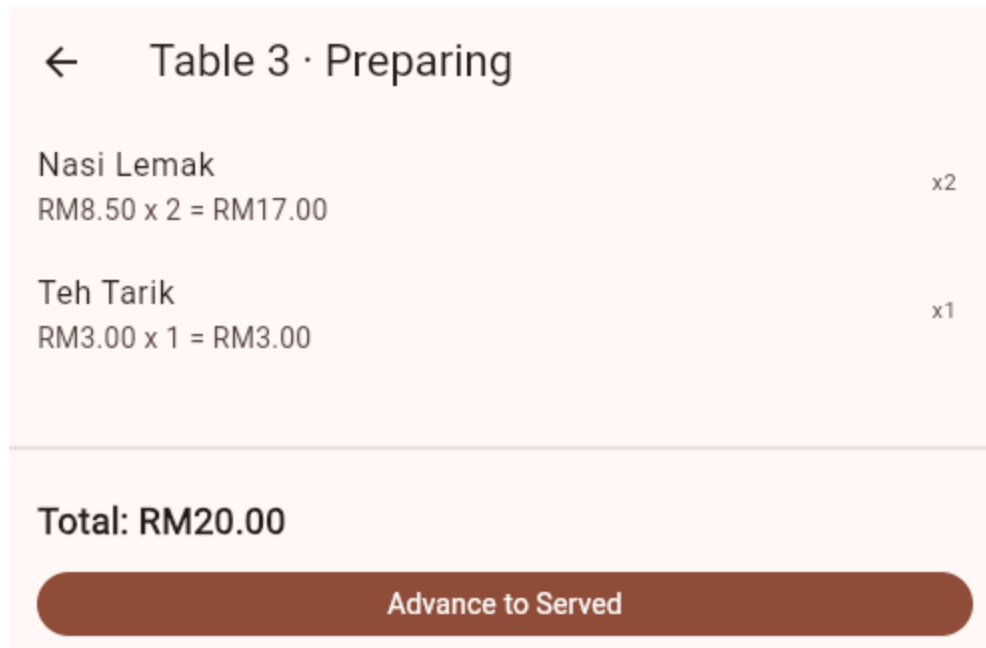


Figure 6 New Order screen

5.0 CONCLUSION

The testing effectively applied a complete Restaurant Waiter Order System covering all needed CRUD operations for both the menu and the order lifecycle. The Menu module supports creating, viewing, editing, deleting, and toggling availability of menu items, while the Orders module supports creating an order with multiple line items, filtering orders by status, advancing an order through its full lifecycle (Pending, Preparing, Served, Paid), editing quantities while an order is still Pending, and canceling a Pending order.

A real cloud backend is used with Row Level Security to ensure that all data persists across applications, restaurants and devices, as required by the rubric. The name and price snapshot pattern in order_items also ensured past orders remain accurate even after a menu item's details are later changed or the item is deleted, which was verified during testing.

Overall, the application meets the scope defined in the requirements, which are five screens, two top-level tabs, no authentication, and full CRUD coverage while remaining simple and easy to demonstrate.